

L1: Kernel Acceptance 仕様と検証アーキテクチャ

An Independently Verifiable Foundation for Lean Certificates

Status Audited design specification

Version 0.3.0

Date 2026-08-09

本書は TheoremForces の assurance model のうち、最下層かつ不可欠な **L1: kernel acceptance** だけを独立して定義する。Lean kernel による型検査、exact artifact と対象定理の拘束、axiom policy、fail-closed な判定、証明書 schema、独立検証、脅威モデル、および実装 gate を規定する。

0.3.0 security revision: hostile Lean process を acceptance authority から外し、trusted Challenge + Comparator、sandbox 外 proof export、Lean checker + independent checker quorum、cycle-free signed envelope、exact artifact manifest、production Gate 0–C を normative に固定した。Gate C 完了まで L1 issuance は無効である。

TheoremForces Project

<https://theoremforces.org/>

Independent project. Not affiliated with, endorsed by, or sponsored by Lean FRO, the Lean project, or the mathlib maintainers.

要旨

L1 は「提出された文字列がもっともらしい」ことではなく、固定された環境において、指定された Lean declaration が trusted Challenge と一致し、exported proof term を official Lean checker と independent checker の quorum が受理したことを表す。ただし kernel は、import 済みの定数や公理を前提として導出可能性を判定するため、受理そのものと axiom policy への適合は区別しなければならない。

TheoremForces の operational L1 は、checker quorum に加えて、exact source hash、environment digest、target declaration、依存閉包、axiom report、実行状態を同一 certificate に拘束する。受理経路が一つでも不明なら L1 を発行しない。第三者は署名や UI を信用せず、artifact と環境を取得し、同じ判定を再実行できる。本書はこの最小構成を検証可能な仕様として提示する。

Abstract

L1 denotes a narrow claim: in an identified Lean environment, the exported proof term associated with a trusted-challenge-bound declaration was accepted by both the pinned Lean checker and an independent checker. This is a claim of derivability relative to the imported environment, not a claim that the formal statement matches an informal intention, is novel, useful, axiom-free, or authored by a particular person.

TheoremForces operationalizes L1 by binding trusted challenge comparison and checker-quorum acceptance to an exact source artifact, exactly one target declaration, environment digest, primitive axiom set, certificate dependency closure, resource outcome, and an append-only certificate record. The design is fail-closed: an ambiguous target, incomplete axiom report, artifact mismatch, timeout, or infrastructure error cannot be represented as acceptance. An independent verifier can reconstruct and replay the judgment without trusting the website.

Normative language and scope

The words **MUST**, **MUST NOT**, **SHOULD**, and **MAY** express requirements in this design specification. Version 0.3 normative correction, Sections 3–10, requirement catalog, Gate 0–C, versioned schema / golden vectors / adversarial fixtures are normative unless explicitly marked informative. The document is version-neutral with respect to Lean and Mathlib: every concrete certificate identifies its own environment rather than relying on a moving notion of “current.”

目次

1	Version 0.3 normative security correction	1
1.1	Hostile Lean and checker quorum	1

1.2	Normative requirement catalog	1
1.3	Fixed identity and canonicalization decisions	4
1.4	Cycle-free certificate and signature profile	4
1.5	Artifact manifest and safe materialization	4
1.6	Attempt state and acceptance transaction	5
1.7	Cumulative transparency and dependency state	5
1.8	Production gates	5
2	L1 が解く問題	6
2.1	最小の公開主張	6
2.2	保証すること、しないこと	6
2.3	設計原則	7
3	Lean の trust boundary	7
3.1	elaboration と kernel checking	7
3.2	trusted computing base	7
3.3	公理に相対的な受理	8
4	L1 acceptance の形式仕様	8
4.1	入力と judgment	8
4.2	artifact identity	9
4.3	environment identity	9
4.4	target binding	10
4.5	複数定理 UX は L1 の外に置く	10
5	axiom policy	11
5.1	三段階の判定	11
5.2	certificate-backed assumption	11
5.3	推奨 policy profile	12
5.4	policy versioning	12
6	判定 pipeline と状態機械	13
6.1	fail-closed pipeline	13
6.2	結果 taxonomy	13
6.3	resource envelope	14

7	L1 certificate schema	15
7.1	必須 field	15
7.2	canonical JSON 例	17
7.3	署名対象と transparency	18
8	独立 verifier	18
8.1	三つの検証 mode	19
8.2	verifier algorithm	19
8.3	determinism と許容差	20
9	脅威モデル	20
9.1	保護対象と攻撃者	20
9.2	sandbox と logical soundness の分離	21
9.3	residual risk	21
10	reproducibility と migration	21
10.1	三つの時間軸	21
10.2	artifact retention	22
10.3	environment epoch の lifecycle	22
11	testing と observability	22
11.1	verification corpus	22
11.2	integrity SLO と運用 metric	23
12	実装 roadmap	24
12.1	Phase 0: claim freeze と fixture (0–30 日)	24
12.2	Phase 1: certificate と replay (31–60 日)	24
12.3	Phase 2: production hardening (61–90 日)	25
12.4	ownership	25
13	決定事項と非規範的 research questions	25
13.1	本仕様で固定する決定	25
13.2	Accepted の意味を変えない research questions	26
14	結論	26
付録 A	minimal judge wrapper 例	27

付録 B	state transition table	28
付録 C	用語集	28
付録 D	参考文献と一次資料	29

1 Version 0.3 normative security correction

NO-GO until Gate C

通常の lean process と human-readable axiom output は hostile proof marketplace の acceptance authority にできない。Version 0.2 で未決定だった type serialization、canonical JSON、signature、artifact root、checker boundary、registry retention は本節で固定する。本節と後続節が衝突する場合は本節を優先する。Gate 0-C 完了前は site を deploy してもよいが、新規 **L1 accepted** issuance は MUST remain disabled であり、historical v1 record は legacy-incomplete と表示する。

1.1 Hostile Lean and checker quorum

User-controlled Lean source、macro、command、tactic、plugin、import、.olean は hostile input である。debug.skipKernelTC、unchecked environment mutation、malicious meta-program を含む通常 build の exit 0 は builder evidence にすぎず、L1 を単独で成立させない。Issuer から分離した verifier domain は次をすべて MUST verify する。

1. Organizer-controlled trusted Challenge artifact digest と submitted declaration の equality を pinned Comparator で確認する。
2. Builder sandbox から proof export を取り出し、sandbox 外の no-user-code verifier へ渡す。
3. Pinned official Lean checker と、異なる implementation / code base の independent checker の両方で export を検査する。
4. Type、value、universe parameters、declaration kind、proof export、transitive axiom closure の digests を再計算する。
5. 一方でも reject、crash、timeout、unsupported、diverge なら accepted にせず、rejected または indeterminate とする。

Initial independent checker profile は policy で pin した nanoda-compatible checker とする。対象 feature が未対応なら quorum を弱めず issuance を止める。Comparator、Lean checker、exporter、independent checker の source commit、binary digest、build provenance を logical environment manifest に含める。Lean の untrusted-proof validation guidance と Comparator model を trust boundary の根拠とする [4, 5]。

1.2 Normative requirement catalog

各 requirement は repository の machine-readable catalog に同じ ID、owner、conformance test、release evidence URI を持つ。Unknown、skipped、flaky、manual-only は PASS ではない。

ID	Normative requirement
L1-CONF-001	全 MUST に stable ID、automated test、versioned fixture、release evidence が存在する。

ID	Normative requirement (continued)
L1-ACC-001	Accepted は本 catalog の必須 subpredicate の conjunction のみであり、missing / unknown は fail-closed。
L1-TRUST-001	Hostile build は non-root、no-secret、read-only input、fresh workspace、default-deny egress、zero capability、seccomp、PID / mount / user namespace または microVM 内だけで実行する。
L1-VER-001	debug.skipKernelTC、unchecked addDecl、malicious meta-code fixture は accepted にならない。
L1-VER-002	Trusted Challenge digest と Comparator equality が必須。Statement substitution は reject する。
L1-VER-003	Lean checker と independent checker quorum が PASS。一方の reject / crash / unsupported は非 accepted。
L1-VER-004	Sandbox 外で validated proof export を検査し、その digest を CertificateCore に拘束する。
L1-TGT-001	Challenge artifact / statement は organizer-controlled CAS object として dispatch 前に固定する。
L1-TGT-002	Environment delta 上の user target は exactly one。Macro-generated second declaration、opaque、axiom、run_cmd injection を reject する。
L1-TGT-003	Fully qualified name、kind、universe parameters、type / value / proof-export digest を記録し replay で一致させる。
L1-ART-001	Raw upload、decoded entrypoint、canonical manifest、bundle root、generated source を別 digest / insert-only CAS object として保存する。
L1-ART-002	Canonical path profile を固定し、absolute / dot / dot-dot / backslash / NUL / case-fold collision / symlink / hardlink / special file を reject する。
L1-ART-003	Build 中 input は immutable で pre / post bundle root が一致する。
L1-ENV-001	Toolchain、imports、binary、image platform、command、option、SBOM、provenance の全 identity を manifest 化する。
L1-ENV-002	Moving tag、network resolution、unmanifested file access、shared mutable cache を禁止する。
L1-ENV-003	Known-unsound / advisory-blocked epoch では exact pin にかかわらず issuance を停止する。
L1-AX-001	Proof export から structured transitive axiom closure を verifier domain で独立再計算する。
L1-AX-002	Primitive baseline は exact declaration identity、kind、type digest、origin module digest で固定する。

ID	Normative requirement (continued)
L1-AX-003	sorryAx、compiler trust、native computation 由来、custom / unknown assumption を deny する。
L1-REG-001	Certified dependency は real proof export または全 upstream checker receipt、type / value / export digest に拘束する。
L1-REG-002	DAG、issuance-before dependency checkpoint、status-at-checkpoint、current status、taint closure を検証する。
L1-CERT-001	Strict JSON Schema、RFC 8785 JCS、I-JSON constraints、cross-language golden vectors を使う。
L1-CERT-002	CertificateCore、IssuanceAttestation、LogReceipt を分離し、Ed25519 domain-separated non-self-referential envelope を使う。
L1-CERT-003	Required field 欠落、unknown field、duplicate key、bad Unicode、bad digest / timestamp / size を reject する。
L1-LOG-001	Cumulative log inclusion / consistency proof、signed checkpoint、two independent witness receipts を metadata verifier が検証する。
L1-KEY-001	Key validity、algorithm、role、rotation、revocation を issuance checkpoint に対して検証する。
L1-RUN-001	Limit と actual CPU / wall / RSS / PID / file / disk / inode / output / network、exit / signal / OOM / timeout / truncation を記録する。
L1-RUN-002	Exit 0、expected structured result exactly once、no truncation、全 checker PASS が accepted の必須条件。
L1-FAIL-001	accepted rejected indeterminate と versioned reason precedence を固定し、attempt receipt と certificate を分ける。
L1-FAIL-002	Terminal transition は conditional DB write で atomic かつ一度。Duplicate / late callback は state を変えない。
L1-REP-001	Verifier default は non-executing metadata mode。--replay は explicit opt-in と強制 sandbox を要求する。
L1-RET-001	Accepted certificate の全 replay object、failure attempt、event、receipt を lifetime 中取得可能にする。欠落時は replay-unavailable event を追記する。
L1-UNI-001	Source bytes は非正規化で hash し、UI は bidi / control / default-ignorable を可視化して escaped byte view を提供する。
L1-OPS-001	False acceptance、checker divergence、artifact mismatch、log fork、key incident で 5 分以内に automatic issuance pause。
L1-OPS-002	Gate C の 30 日観測完了まで production L1 issuance を禁止する。

ID	Normative requirement (continued)
L1-TEST-001	Adversarial corpus、clean-host replay、second-operator replay を release gate にする。

1.3 Fixed identity and canonicalization decisions

Pretty-printer output を target identity に使わない。Target identity は pinned exporter が生成する versioned, length-delimited kernel declaration record の exact bytes とする。Record は fully qualified name、declaration kind、universe parameters、kernel Expr type、kernel Expr value / opaque proof reference、referenced declaration identities を含み、`target.type_sha256`、`target.value_sha256`、`target.proof_export_sha256` を別に計算する。Exporter format / binary digest は environment manifest に拘束する。Format 変更は new schema / environment epoch と golden vectors を要求する。

JSON は RFC 8785 JCS と I-JSON constraints に固定する。Duplicate object key、lone surrogate、non-finite number、negative zero、schema が許可しない null / unknown field を reject し、文字列を NFC 等へ暗黙変換しない。Digest は lowercase sha256: + 64 lowercase hex。Signature algorithm は Ed25519 / RFC 8032、signature encoding は base64url without padding とする [6, 7]。

1.4 Cycle-free certificate and signature profile

L1 public object は **CertificateCore**、**IssuanceAttestation**、**LogReceipt** の三層である。Core は artifact、environment、target、axiom / certificate dependency closure、resource outcome、decision を含む。Issuance payload は core digest、immutable JobExpectation / ResultAttestation digests、verifier quorum、issuer key ID、`issued_at`、policy digest を含む。Signatures は payload 外側に置く。LogReceipt は core digest と issuance digest を leaf へ commit し、core / issuance の hash 対象には入れない。

$$d_C = \text{SHA256}(\text{UTF8}(\text{TF-CERT-CORE-V2}) \parallel 00_{16} \parallel \text{JCS}(C)), \quad (1)$$

$$d_I = \text{SHA256}(\text{UTF8}(\text{TF-ISSUANCE-V1}) \parallel 00_{16} \parallel \text{JCS}(I)), \quad (2)$$

$$m_I = \text{UTF8}(\text{theoremforces.l1.signature.v2}) \parallel 00_{16} \parallel \text{JCS}(I). \quad (3)$$

Receipt / witness の追加は d_C, d_I を変えてはならない。Issuer、builder、verifier、log key は role-separated key ID、validity interval、rotation overlap、revocation event を持つ。Compromise 時は historical validity と current trust state を分離し、影響 prefix と downstream taint を append する。

1.5 Artifact manifest and safe materialization

`artifact.upload_raw_sha256`、`artifact.entrypoint_raw_sha256`、`artifact.manifest_sha256`、`artifact.bundle_root_sha256` は別 field とする。Canonical manifest は server-generated portable ASCII path の昇順で、regular file raw digest、size、fixed mode を commit する。

$$d_B = \text{SHA256}(\text{UTF8}(\text{theoremforces.artifact.v1}) \parallel 00_{16} \parallel \text{JCS}(\text{CanonicalManifest})). \quad (4)$$

Archive は compressed bytes を CAS digest / size で検査してから、entry count、expanded bytes、depth、compression ratio cap の下で fresh directory へ extract する。URI は `cas://sha256/...` と allowlisted HTTPS mirror のみを許し、redirect、private IP、file:、cloud metadata endpoint を拒否する。Verifier metadata mode は artifact を実行しない。

1.6 Attempt state and acceptance transaction

Web / API は dispatch 前に attempt ID、job nonce、challenge / input / generated source / dependency / environment / policy digests を immutable JobExpectation として commit する。Builder は run / attempt number、同じ digests、pre / post root、resource outcome を signed ResultAttestation で返す。Callback credential だけを evidence としない。

State は received → dispatched → result-attested → independently-verified → blobs-durable → log-integrated → published の単調遷移とする。Conditional terminal update、certificate row、reputation credit、transactional outbox は一つの idempotency boundary で exactly once にする。Object store / queue / log の eventual work は outbox event で継続し、published 前は public accepted と数えない。Retry は新 attempt、reverification は新 event であり historical row を上書きしない。

1.7 Cumulative transparency and dependency state

LogLeafV2 は schema = theoremforces.l1-log-leaf/2、core_sha256 = d_C、issuance_sha256 = d_I の三 field だけを持つ object とする。Merkle leaf input は $L = \text{JCS}(\text{LogLeafV2})$ 、leaf は $\text{SHA256}(0x00 \parallel L)$ 、internal node は $\text{SHA256}(0x01 \parallel \text{left} \parallel \text{right})$ とし、odd node を複製しない。Signed checkpoint は log ID、tree size、root、timestamp、algorithm、key ID、maximum merge delay を含む。Receipt は inclusion proof、直前 checkpoint からの consistency proof、有効な log signature、少なくとも二つの independent witness receipts を持つ。MMD 内に統合される前は pending-log であり certificate ではない [8]。

Registry は append-only **Registry Event Log** と、event prefix から導出した **Registry State Checkpoint** に分ける。activeAtIssue(c,R_issue) と currentStatus(c,R_now) を別 field / query にする。Later invalidation は historical acceptance を改変せず、全 downstream へ deterministic taint event を追加する。Stub は elaboration cache にすぎず、accepted issuance は全 upstream proof export を topological order で checker quorum に通す。UI は primitive assumptions、certified assumptions、stub rebuild、full-chain replay を別々に表示する。

1.8 Production gates

Gate 0 - Spec freeze

本 catalog、JSON Schema、JCS / signature / Merkle golden vectors、checker profile、failure enum、key policy、全 ADR を固定し、accepted の意味に関する open decision を 0 件にする。

Gate A - Adversarial CI

Kernel-skip、malicious meta、statement substitution、custom / native axiom、stale .olean、duplicate JSON、bad Unicode、path collision、symlink、archive bomb、callback replay、resource overrun、log fork の corpus を全件 reject する。

Gate B - Independent replay

Staging sample の 100% を clean host / network off で再構成し、Comparator、Lean checker、independent checker、type / value / export / axiom digest が一致する。Second operator が同じ result を得る。

Gate C - Production readiness

30 日間、必須 evidence 100%、sampled replay 100%、false acceptance 0、checker divergence 0 を維持し、issuance kill-switch、key revocation、restore、log fork、downstream taint drill を完了する。

2 L1 が解く問題

2.1 最小の公開主張

AI と tactic が生成する Lean code は、人間が全行を理解しなくても kernel で検査できる。しかし “build succeeded” というログ断片だけでは、何が検査されたかを第三者が同定できない。同じ proof body でも、import、Lean / Mathlib revision、対象 theorem、許可された axiom が異なれば、主張の意味は変わる。L1 は次の一文を、機械で検証できる record に変換する。

L1 claim

Certificate が指す exact artifact と exact environment から構成された declaration $q : T$ が trusted Challenge と Comparator で一致し、その exported proof term t を sandbox 外の official Lean checker と independent checker が T の inhabitant として受理し、かつ公開された L1 policy の必須条件がすべて成立した。

この主張は小さい。しかし certificate の上位層、再検証、review、citation、migration のすべては、この最小事実が曖昧でないことに依存する。L1 は badge の色ではなく、再実行可能な判定 protocol である。

2.2 保証すること、しないこと

L1 が保証する	L1 単独では保証しない
指定環境に対する formal derivability	informal statement と研究者の意図の一致
exact artifact と target declaration の同一性	数学的新規性、重要性、読みやすさ
記録された axiom report と policy 判定	許可公理の哲学的妥当性
失敗と infrastructure error の非混同	人間による著者性、AI 使用の申告
第三者が replay できるための最小 material	将来の Lean / Mathlib でも通ること

したがって L1 の表示は “verified mathematics” の包括的宣言ではない。とりわけ、形式化された命題が空虚である、仮定が強すぎる、別の定理を証明している、といった問題は L1 では検出しない。それらは intent review や curation など上位 layer の責務である。

2.3 設計原則

1. **Small trusted core:** 論理的受理は Lean kernel に帰属させる。
2. **Exact binding:** source、environment、target、policy、結果を hash で一体化する。
3. **Fail closed:** 不明、欠落、timeout、parser failure を受理へ昇格させない。
4. **Replay over reputation:** platform の評判ではなく、第三者の再実行を最終検証手段にする。
5. **Historical truth:** 過去の受理 record を上書きせず、再検証を別 event として追加する。
6. **Separation of claims:** kernel acceptance、axiom compliance、運用成功、署名検証を別 field にする。

3 Lean の trust boundary

3.1 elaboration と kernel checking

Lean source は parser、macro、elaborator、tactic framework を通り、明示的な declaration と proof term に変換される。tactic は証明探索を助けるが、最終的な term は kernel が型検査する。Lean reference は kernel、公理、proof validation の役割を個別に記述している [1, 2, 3, 4]。

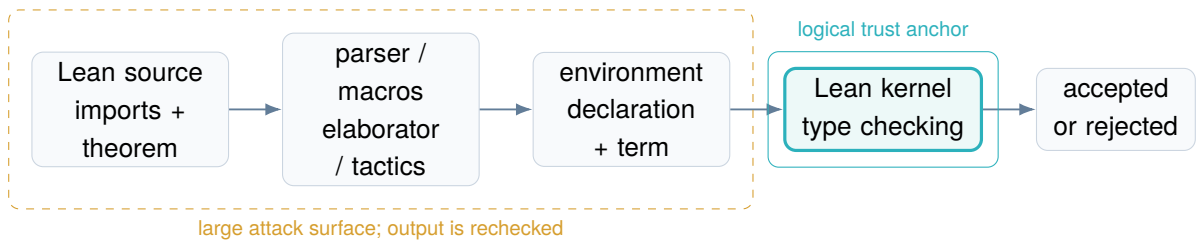


図 1: L1 における生成経路と論理的 trust boundary

図 1 は elaborator や tactic が無関係だという意味ではない。それらの bug は crash、resource exhaustion、誤った error、意図しない term の生成を起こしうる。ただし、kernel が健全であり、不正な axiom や定数を環境へ導入していない限り、誤った型の term を proof として通すことはできない。L1 はこの構造を利用しつつ、環境の作り方と axiom を明示的に policy の対象にする。

3.2 trusted computing base

L1 の trusted computing base (TCB) は “kernel binary だけ” ではない。現実の再検証では、次が結果を左右するため、certificate はそれらを識別しなければならない。

構成要素	L1 での役割	拘束方法
Lean kernel / runtime	declaration の型検査を実行	release、commit、binary / image digest
import 済み environment	定数、theorem、axiom を供給	lockfile、module set、dependency digest
Mathlib と追加 package	大規模な既存 theorem 群	exact revision と package manifest
judge wrapper	target を構成し検査命令を実行	wrapper version と artifact hash
axiom analyzer	使用 axiom の閉包を報告	tool version、生出力、strict parser
certificate encoder	field を canonicalize して署名対象化	schema version、canonicalization version

OS、container runtime、CPU は論理規則そのものではないが、replay の再現性と supply-chain integrity に関わる。したがって platform image digest、architecture、runner identity も operational metadata として残す。

3.3 公理に相対的な受理

Kernel acceptance は「公理を一切使わない」を意味しない。Lean environment に declaration として存在する axiom は、型を持つ定数として proof term から参照できる [3]。よって、 $\Delta \vdash t : T$ は environment Δ に相対的な judgment である。

重要な非同値

kernel accepted $\not\equiv$ axiom-policy compliant.

sorry、custom axiom、classical choice、quotient 関連原理などの扱いは policy に依存する。L1 certificate は kernel status と policy status を別々に記録し、両方が PASS の場合だけ operational acceptance を発行する。

4 L1 acceptance の形式仕様

4.1 入力と judgment

L1 判定の入力 tuple を

$$I = (C, A, E, q, P, \Gamma, R, X)$$

とする。 C は trusted Challenge artifact、 A は exact artifact、 E は environment descriptor、 q は target declaration、 P は primitive axiom / namespace policy、 Γ は certificate registry checkpoint、 R は resource envelope、 X は dispatch 前の immutable JobExpectation である。Hostile builder は A を E 内で処理し proof export x を生成するが、その process 内の Δ, T, t を authority としない。

Verifier domain の checker judgment は次である。

$$V(x, E, q) = \text{PASS} \iff \text{LeanChecker}(x) = \text{PASS} \wedge \text{IndependentChecker}(x) = \text{PASS}$$

TheoremForces の operational L1 acceptance は、より強い合成 predicate とする。

$$\begin{aligned}
 L1(I) = \text{PASS} \iff & \text{ChallengeMatch}(C, q) \wedge \text{ArtifactMatch}(A) \wedge \text{EnvironmentMatch}(E) \\
 & \wedge \text{ExpectationMatch}(X) \wedge \text{Comparator}(C, q) = \text{PASS} \\
 & \wedge \text{TargetBound}(q, x) \wedge \text{UserTargetCount}(A) = 1 \\
 & \wedge V(x, E, q) = \text{PASS} \\
 & \wedge \text{PrimitiveAxiomPolicy}(x, E, P) = \text{PASS} \\
 & \wedge \text{CertifiedDependencies}(x, E, \Gamma) = \text{PASS} \\
 & \wedge \text{ResourceOutcome}(R) = \text{completed} \\
 & \wedge \text{EvidenceComplete}(I) \\
 & \wedge \text{DurableAndLogIntegrated}(I).
 \end{aligned}$$

Normative rule L1-1: conjunctive acceptance

Issuer は上記 predicate の全 conjunct を機械的に確認した場合にのみ `l1.status = "accepted"` を発行しなければならない。部分成功を `accepted` と表現してはならない。個々の sub-status は保持する。

4.2 artifact identity

Artifact は proof body だけではなく、判定に入力された完全な byte sequence である。最低限、次を含む。

- import list、namespace、variables、statement、proof body を含む生成済み Lean file、
- judge が追加する wrapper / prelude の version と exact bytes、
- package manifest、lockfile、Lake configuration、追加 source files、
- source tree manifest (path、size、content hash) と root hash。

Unicode normalization、改行変換、末尾空白削除を暗黙に行ってはならない。canonical representation を作る場合も、raw bytes の hash を必ず残す。UI に表示した reconstruction ではなく judge が読んだ bytes が基準である。

4.3 environment identity

Environment descriptor は、人が読む version label と機械が検証する immutable identity を分ける。

field	意味	要件
lean.version, lean.commit	compiler / kernel source identity	MUST
mathlib.commit	Mathlib revision	MUST if imported
manifest.sha256	full dependency resolution	MUST
image.digest	executable filesystem image	MUST for hosted run
architecture	CPU / OS execution target	MUST
imports	target file の import list	MUST
environment.epoch	platform policy 上の immutable epoch	MUST

“Lean 4” や “latest Mathlib” は L1 identity として不十分である。tag が movable でなくても、transitive dependency と build option を復元できなければ replay 可能とはいえない。

4.4 target binding

L1 の原子単位を **one attempt = one theorem** とする。user payload は top-level の theorem / lemma を厳密に一つだけ含まなければならない。複数の declaration のうち一つだけを target として選ぶ方式は採用しない。使われない第二 theorem も multiple_target_declarations として拒否する。proof 内の have、let、local function は単一 proof term に閉じるため許可できるが、再利用する補助定理は別 attempt で certificate 化しなければならない。server-generated wrapper declaration は count から除外し、その exact bytes と template version を artifact に拘束する。

judge は唯一の target の fully qualified name、expected type hash、source span、declaration kind を記録する。

Normative rule L1-2: single user target

Issuer は user-supplied top-level target declaration 数が一つであり、その selector が environment 内でただ一つの declaration に解決されることを確認しなければならない。0 件、2 件以上、名前の shadowing、expected type の不一致を受理してはならない。

可能なら judge wrapper が新しい namespace と衝突しない generated identifier を用意し、対象 statement と proof body を直接結合する。既存 theorem と同名の declaration を偶然参照して通過する経路をなくす。

4.5 複数定理 UX は L1 の外に置く

UI / API は複数 theorem の一括入力を提供してよい。ただし batch orchestrator が入力を N 個の独立 artifact に分け、 N 個の L1 attempt、resource envelope、result、certificate を生成する。batch / collection manifest は結果の集約であり、L1 certificate ではない。部分成功を許容し、失敗した job だけを retry する。この境界により、一つの theorem の失敗で別 theorem の historical acceptance が曖昧にならず、cache、quota、citation、dependency edge も theorem 単位になる。

Normative rule L1-3: no atomic multi-target acceptance

Issuer は複数 theorem を一つの `l1.status = "accepted"` に畳み込んで서는ならない。collection の status は各 child attempt への参照と集計値だけを持ち、独自の kernel acceptance を主張しない。

5 axiom policy

5.1 三段階の判定

axiom analyzer は、kernel checking 後に target の transitive dependency closure を調べ、**primitive assumptions** と **certificate-backed assumptions** に分離する。前者だけを **allowed**、**denied**、**unknown** に分類し、unknown は fail-closed とする。後者は名前ではなく certificate registry によって検証する。

1. target declaration から参照定数を再帰的に列挙する。
2. opaque / theorem boundary を含め、primitive axiom 集合と certificate dependency 集合を得る。
3. primitive assumption は canonical axiom identifier と environment-local identifier を対応付ける。
4. certificate assumption は registry object、type hash、logical environment digest と対応付ける。
5. primitive policy と certificate chain の両方が PASS のときだけ operational policy status を PASS にする。

Lean の `#print axioms` は debugging 用の補助 evidence として保存してよいが、L1 acceptance authority にしてはならない。Pinned proof exporter と二つの checker が proof export から structured transitive closure を再計算し、human-oriented stdout の parse failure、format drift、truncation は indeterminate とする [4]。

5.2 certificate-backed assumption

一度 L1 accepted された theorem は、downstream elaboration cache として server-generated な opaque / axiom stub を利用してよい。ただし stub build は certificate assumptions に相対的な結果であり、L1 issuance の checker path は全 upstream proof export を topological order で再検査する。概念的 cache は次の形である。

```
namespace TheoremForces.Certified
-- Generated only after resolving certificate sha256:<c>.
axiom c_<certificate_hash> : <exact certified statement>
end TheoremForces.Certified
```

これは「user axiom を許可する」ことではない。namespace の文字列一致では受理せず、resolver が certificate hash から生成した module hash、stub の canonical type hash、発行時の logical environment digest を照合する。user-defined axiom、同名 declaration、改変された stub は拒否する。accepted theorem の historical artifact は保持し、full verifier は certificate chain を再帰的に replay する。一つでも certified dependency があれば UI / API / CLI は axiom-free または unconditional proof と表示しない。

primitive axiom 集合を $P_E(t)$ 、certificate-backed dependency 集合を $C_E(t)$ とする。certificate import の条件は

$$\forall c \in C_E(t), \quad \text{ActiveAtIssue}(c, \Gamma_{\text{issue}}) \wedge \text{TypeHashMatch}(c) \wedge \text{LogicalEnv}(c) = E,$$

かつ $C_E(t)$ の transitive graph が acyclic で、全 upstream certificate に同じ条件が成り立つことである。発行時 validity と current status を分離する。Dependency が invalidated / retracted-by-policy になった場合、downstream certificate を削除せず tainted event を追加し、再発行を停止して影響 closure を再検証する。

Normative rule L1-4: same logical environment

Certificate-backed assumption は、Lean version、Mathlib / package manifest、primitive axiom policy、wrapper / analyzer rule を含む `logical_environment_digest` が importing attempt と一致する場合にだけ使用できる。human-readable label や registry namespace の一致だけで cross-environment import を許可してはならない。

5.3 推奨 policy profile

category	default	理由と表示
sorryAx / synthetic sorry	deny	未完了証明を完全な proof と表示しない
user-defined axiom	deny	明示的 review なしの仮定導入を防ぐ
Lean / Mathlib standard axioms	explicit allowlist	profile と environment ごとに名前を固定
unrecognized declaration	unknown → deny	analyzer / package drift を安全側で扱う
unsafe implementation detail	separate operational check	論理公理と runtime risk を混同しない

“axiom-free” という label は、使用集合が空であることを再検証可能に示せる場合だけ使う。標準的な古典論理の原理を許可する profile は、単に “L1 accepted” とし、実際の axiom set を併記する。

5.4 policy versioning

Axiom policy は immutable object として content-addressed storage に置く。certificate は `policy.id`、`policy.sha256`、判定時の classification を持つ。後日 policy が変わっても、historical record は変えず、新 policy による再評価 event を追加する。これにより “当時は受理、現在 profile では拒否” を正確に表現できる。

accepted theorem を追加するたびに primitive axiom policy を更新してはならない。logical base

epoch E と primitive policy P は低頻度で versioning し、same- E certificate の集合は append-only registry checkpoint $\Gamma_0, \Gamma_1, \dots$ として高頻度に更新する。import は exact certificate hash を指すため、checkpoint が進んでも既存 certificate の意味は変わらない。別 E へ移すときは migration replay と新 certificate が必要である。

頻繁に変わるのは「公理 policy」ではなく registry root

新しく accepted された命題は primitive allowlist に追加しない。certificate-backed assumption として registry に追記する。したがって standard axiom profile の version は安定し、頻繁な更新は content-addressed checkpoint root に局所化される。

6 判定 pipeline と状態機械

6.1 fail-closed pipeline

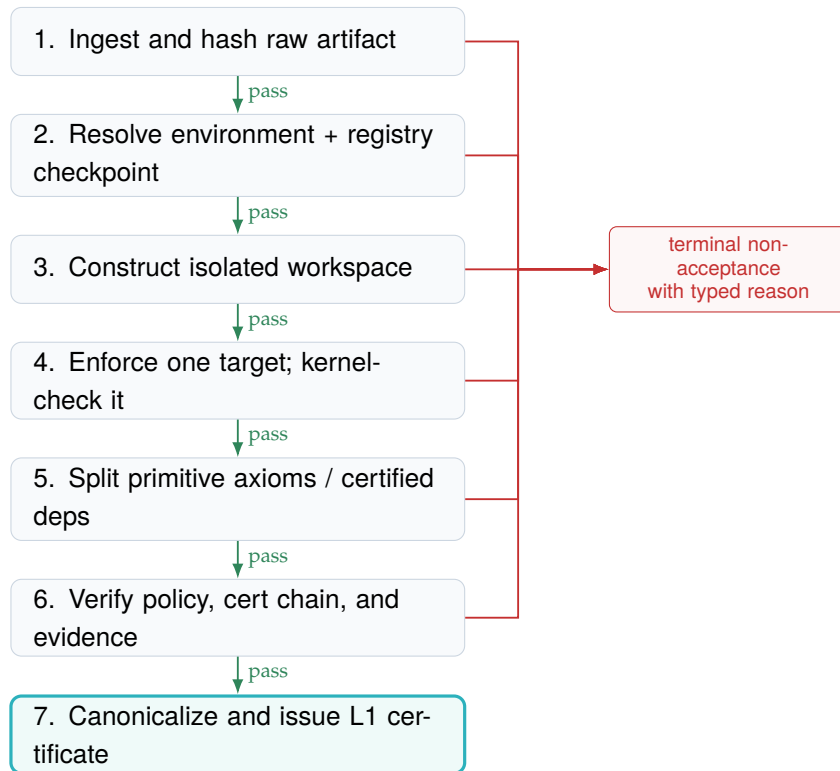


図 2: L1 judge の一方向 pipeline。失敗後に受理経路へ復帰しない。

各 stage は input / output digest、開始・終了時刻、runner identity、typed result を記録する。retry は同じ run の書換えではなく新しい attempt とし、certificate が参照する attempt を一意にする。

6.2 結果 taxonomy

result code	L1	意味
accepted	accepted	kernel と全 operational predicate が PASS
parse_or_ elaboration_failure	not accepted	source から target declaration を構成できない
kernel_or_type_error	not accepted	target term が期待する type を持たない
multiple_target_ declarations	not accepted	user payload に top-level theorem / lemma が 2 件以上
axiom_policy_rejection	not accepted	deny または unknown axiom を検出
certificate_ dependency_invalid	not accepted	certificate、type hash、status、closure が不正
certificate_ environment_ mismatch	not accepted	dependency と importing attempt の logical environment が不一致
certificate_ dependency_cycle	not accepted	certificate dependency graph に cycle を検出
target_mismatch	not accepted	name、type hash、declaration identity が不一致
artifact_mismatch	not accepted	judge input と certificate 対象の bytes が不一致
resource_exhausted	indeterminate	timeout、memory、disk、process quota 超過
infrastructure_error	indeterminate	image、storage、scheduler、network 等の障害
evidence_incomplete	indeterminate	axiom report、manifest、log 等の必須証拠が欠落

not accepted は反例や数学的偽を意味しない。**indeterminate** は再試行可能だが、accepted と表示してはならない。UI、API、export、analytics は同じ taxonomy を使い、HTTP status や自由文 log から意味を再推定しない。

6.3 resource envelope

CPU、wall time、memory、file size、process count、output bytes に上限を設ける。上限は certificate に記録し、超過を proof failure と区別する。resource policy の目的は logical soundness そのものではなく、公平性、可用性、abuse containment である。sandbox が破られても kernel acceptance record を偽造できないよう、証明書 signing と public ledger ingestion は judge process から分離する。

7 L1 certificate schema

7.1 必須 field

L1 certificate は self-contained な metadata と content-addressed artifact への参照を持つ。最低限の schema を表 3 に示す。表中の CertificateCore field path は `core.` prefix を省略し、`issuance_attestation` と `log_receipt` だけは envelope root からの path を記す。

表 3: L1 certificate の normative field

field	type	requirement
<code>schema, certificate_id</code>	string	core schema version と globally unique ID
<code>attempt_id,</code> <code>target.challenge_</code> <code>artifact_sha256</code>	string / digest	immutable attempt と organizer-controlled Challenge
<code>artifact.upload_raw_</code> <code>sha256</code>	digest	transport で受け取った exact bytes
<code>artifact.entrypoint_raw_</code> <code>sha256</code>	digest	decoded Lean entrypoint exact bytes
<code>artifact.manifest_sha256</code>	digest	canonical path / size / mode / file hash manifest
<code>artifact.bundle_root_</code> <code>sha256</code>	digest	domain-separated canonical manifest root
<code>artifact.generated_</code> <code>source_sha256</code>	digest	builder が実際に読んだ generated source
<code>environment.epoch</code>	string	immutable platform environment
<code>environment.</code> <code>logical_digest</code>	digest	Lean、packages、primitive policy、 wrapper rule の identity
<code>environment.</code> <code>manifest_sha256</code>	digest	Lean、packages、options の完全 manifest
<code>environment.image_digest</code>	digest	hosted execution image
<code>target.fqname</code>	string	fully qualified declaration name
<code>target.user_</code> <code>declaration_count</code>	integer	MUST equal 1
<code>target.declaration_kind</code>	enum	MUST equal theorem

field	type	requirement
target.universe_ parameters_format / universe_parameters_ sha256	string / digest	versioned universe parameters
target.type_format / type_sha256	string / digest	versioned target type
target.value_format / value_sha256	string / digest	versioned target value
target.proof_export_ format / proof_export_ sha256	string / digest	versioned proof export
comparator.status	enum	trusted Challenge equality PASS / non-PASS
checkers.items	array	Lean checker と independent checker の identity / result / evidence digest
axioms.primitive_items	array	primitive axiom set と classification
axioms.raw_report_sha256	digest	analyzer の raw evidence
certified_dependencies. items	array	cert / type hash、 constant、 direct / transitive edge
certified_dependencies. registry_checkpoint	digest	判定に使った event-prefix checkpoint
certified_dependencies. closure_sha256	digest	acyclic dependency closure の identity
policy.id, policy.sha256	string / digest	immutable policy identity
run.outcome	enum	accepted core では MUST equal completed
run.limits / usage	object	CPU、 wall、 RSS、 PID、 file、 disk、 inode、 output、 network の limit と actual
run.stdout_sha256 / run.stderr_sha256	digests	完全 log への content address と truncation flag
l1.status, l1.reason	enum	accepted と typed decision reason
issuance_attestation. payload.core_sha256	digest	domain-separated JCS CertificateCore digest
issuance_attestation. payload / signature	objects	core / expectation / result / verifier quorum と Ed25519 envelope

field	type	requirement
log_receipt	object	cumulative inclusion / consistency / checkpoint / witness evidence

7.2 canonical JSON 例

次は三層 topology の縮約例である。掲載した field names とその object topology は normative schema に一致するが、紙幅のため required field の一部を省略している。この紙面例自体を test vector にせず、repository の完全 fixture を JCS / schema / signature / Merkle test に使う。

```
{
  "schema": "theoremforces.l1-certificate/2",
  "core": {
    "schema": "theoremforces.l1-core/2",
    "certificate_id": "tf-l1-01JEXAMPLE",
    "attempt_id": "01JATTEMPT",
    "issued_at": "2026-08-09T12:00:00Z",
    "event_sequence": 42,
    "artifact": {
      "upload_raw_sha256": "sha256:...",
      "upload_raw_uri": "cas://sha256/...",
      "entrypoint_raw_sha256": "sha256:...",
      "entrypoint_raw_uri": "cas://sha256/...",
      "manifest_sha256": "sha256:...",
      "manifest_uri": "cas://sha256/...",
      "bundle_root_sha256": "sha256:...",
      "generated_source_sha256": "sha256:...",
      "generated_source_uri": "cas://sha256/...",
      "proof_export_sha256": "sha256:...",
      "proof_export_uri": "cas://sha256/..."
    },
    "environment": {
      "logical_digest": "sha256:...",
      "manifest_sha256": "sha256:...",
      "image_digest": "sha256:...",
      "architecture": "linux/amd64"
    },
    "target": {
      "challenge_id": "tf-challenge-01JEXAMPLE",
      "challenge_artifact_sha256": "sha256:...",
      "challenge_artifact_uri": "cas://sha256/...",
      "fqname": "TheoremForces.Attempt.solution_target",
      "user_declaration_count": 1,
      "declaration_kind": "theorem",
      "type_sha256": "sha256:...",
      "value_sha256": "sha256:...",
      "proof_export_sha256": "sha256:..."
    },
    "certified_dependencies": {
      "status": "pass",
      "registry_checkpoint": "sha256:...",
      "items": [],
      "closure_sha256": "sha256:..."
    },
    "axioms": {
      "status": "pass",
      "policy_sha256": "sha256:...",
      "primitive_items": [],
      "closure_sha256": "sha256:..."
    },
    "comparator": {"status": "pass", "evidence_sha256": "sha256:..."},
    "checkers": {
      "status": "pass",
      "items": [
        {"role": "lean_checker", "status": "pass"},
        {"role": "independent_checker", "status": "pass"}
      ]
    },
    "run": {"outcome": "completed", "structured_result_count": 1},
    "l1": {"status": "accepted", "reason": "all_predicates_passed"}
  },
  "issuance_attestation": {
    "schema": "theoremforces.l1-issuance-envelope/1",
```

```

    "payload": {
      "schema": "theoremforces.l1-issuance/1",
      "core_sha256": "sha256:...",
      "job_expectation_sha256": "sha256:...",
      "result_attestation_sha256": "sha256:...",
      "issuer": {"key_id": "tf-issuer-2026-01", "algorithm": "Ed25519"},
      "issued_at": "2026-08-09T12:00:00Z"
    },
    "signature": "..."
  },
  "log_receipt": {
    "schema": "theoremforces.l1-log-receipt/2",
    "core_sha256": "sha256:...",
    "issuance_sha256": "sha256:...",
    "leaf_hash": "sha256:...",
    "tree_size": 42,
    "leaf_index": 41,
    "inclusion_path": [],
    "checkpoint": {
      "schema": "theoremforces.l1-log-checkpoint-envelope/2",
      "payload": {
        "schema": "theoremforces.l1-log-checkpoint/2",
        "log_id": "tf-log-v2",
        "algorithm": "rfc6962-sha256",
        "tree_size": 42,
        "root_sha256": "sha256:...",
        "maximum_merge_delay_ms": 300000
      },
      "signature": "..."
    },
    "consistency": {
      "old_tree_size": 41,
      "old_root_sha256": "sha256:...",
      "consistency_path": []
    },
    "witness_receipts": [
      {"schema": "theoremforces.l1-witness-envelope/2",
       "payload": {"witness": {"witness_id": "witness-a"}}, "signature": "..."},
      {"schema": "theoremforces.l1-witness-envelope/2",
       "payload": {"witness": {"witness_id": "witness-b"}}, "signature": "..."}
    ]
  }
}

```

7.3 署名対象と transparency

Ed25519 signature message は前節で定義した m_I 、すなわち `theoremforces.l1.signature.v2` domain、NUL separator、`issuance_attestation.payload` の JCS bytes の順の連結とする。Signature 自身、LogReceipt、witness は signed payload の外側であり自己参照しない。Private signing key は judge / builder container に置かず、separate issuer service が verifier quorum と全 predicate を再確認した後だけ使用する。公開 key の validity、rotation、revocation を append-only event log に残す。署名は issuer を認証するが、checker quorum と artifact replay の代替ではない。

8 独立 verifier

8.1 三つの検証 mode

mode	確認範囲	用途
Metadata verification	strict schema、core / issuance digest、signature、log inclusion / consistency / witnesses	non-executing citation / UI 検査
Historical replay	mandatory sandbox + certificate と同じ environment	元の L1 claim の独立再現
Migration replay	新しい environment / policy	current compatibility の追加評価

Historical replay と migration replay を混同しない。後者が失敗しても historical acceptance は消えず、reverification event に結果を追加する。

8.2 verifier algorithm

```

verify_l1(certificate C):
  validate_schema_strict(C)
  core_sha256 = core_digest(C.core)
  issuance_sha256 = issuance_digest(C.issuance_attestation.payload)
  require C.issuance_attestation.payload.core_sha256 == core_sha256
  verify_ed25519_envelope(C.issuance_attestation)
  verify_key_valid_at_issuance_checkpoint(C.issuance_attestation)
  fetch artifact_manifest, environment_manifest, policy, evidence
  require every fetched object's digest equals the digest in C
  require canonical artifact manifest and bundle root match C.core
  require immutable JobExpectation matches signed ResultAttestation
  resolve certified dependencies at
    C.core.certified_dependencies.registry_checkpoint
  require every certificate was valid at issuance checkpoint
  report current dependency status and taint separately
  require every dependency type/value/export hash matches
  require the transitive certificate graph is acyclic
  require C.log_receipt.core_sha256 == core_sha256
  require C.log_receipt.issuance_sha256 == issuance_sha256
  verify cumulative log inclusion, consistency, checkpoint signature
  require at least two valid independent witness receipts
  require recorded Comparator and both checker receipts are PASS
  if explicit --replay:
    reconstruct only inside mandatory sandbox
    require trusted Challenge Comparator equality
    export and inspect proof outside hostile builder sandbox
    require Lean checker PASS and independent checker PASS
    require type/value/export/axiom/resource digests match C.core
  return an assurance vector, never one ambiguous VERIFIED boolean

```

verifier は network から取得した object を digest 確認前に実行しない。archive extraction の path traversal、symlink escape、decompression bomb を拒否する。signature が正しくても artifact が取得不能なら **metadata-verified / replay-unavailable** とし、full verification と表示しない。

8.3 determinism と許容差

Lean build の wall time や log ordering は完全には deterministic でない。L1 が一致を要求するのは、target identity、kernel result、axiom set、入力 digest である。timestamp、runner ID、性能値は attempt-local metadata とする。同じ入力で論理結果が分岐した場合は severity-1 integrity incident として新規発行を止める。

9 脅威モデル

9.1 保護対象と攻撃者

保護対象は、(1) accepted status の真正性、(2) artifact と environment の同一性、(3) axiom report の完全性、(4) historical ledger の非改変性、(5) judge availability である。攻撃者として malicious submitter、侵害された runner、誤設定された deploy、supply-chain attacker、権限を持つ insider を想定する。

threat	attack	required control
Source substitution	UI 表示と judge input を差し替える	raw-byte hash、manifest、CAS、署名 binding
Target confusion	別 theorem / 既存 theorem を通す	generated namespace、fqname、type hash
Multi-target smuggling	第二 theorem に未監査の前提や副作用を隠す	one user target validator、batch fan-out
Hidden axiom	sorry や custom axiom を依存へ埋める	transitive closure、strict allowlist、raw evidence
Certified namespace spoof	registry にない同名 axiom / stub を作る	server-generated module、cert / type / module hash
Cross-epoch certificate	異なる axiom / package 環境の theorem を流用	logical environment digest equality、migration certificate
Dependency cycle / revocation	循環で検証を逃れる、失効 theorem を継続利用	DAG check、append-only status、downstream taint
Parser drift	axiom output の未知形式を空集合と読む	version pin、strict parse、unknown = reject
Fake callback	runner success を署名なしで ledger へ送る	authenticated channel、ledger-side predicate check
Image substitution	label は同じで binary を交換	immutable digest、registry verification、attestation
Dependency omission	manifest 外から package を読む	offline build、read-only store、network deny

threat	attack	required control
Resource escape	fork bomb、file / network abuse	namespace、cgroup、seccomp、quota、egress policy
Log truncation	error を隠し success 部分だけ保存	complete-log digest、exit status、size-limit outcome
Ledger rewrite	historical failure を success に置換	append-only events、external checkpoint、audit export
Key compromise	偽 certificate を署名	HSM / isolated signer、rotation、revocation、transparency log

9.2 sandbox と logical soundness の分離

Sandbox は infrastructure を守り、他 submission との isolation を提供する。kernel checking は logical claim を検査する。sandbox が強いことは proof の正しさを意味せず、proof が正しいことは arbitrary code execution が安全であることを意味しない。security documentation と judge の実装要件は、この二つの control plane を分けて扱う [11, 10]。

9.3 residual risk

L1 は Lean kernel 自体の soundness bug、hardware fault、compiler / runtime の supply-chain compromise を数学的に排除しない。対策は TCB 縮小、exact version、複数 runner での再検証、公開 artifact、upstream advisory monitoring、incident response である。高価値 certificate には異なる architecture または独立運営者による replay receipt を追加することを SHOULD とする。

10 reproducibility と migration

10.1 三つの時間軸

Certificate の状態は一つの boolean でなく、少なくとも次の時間軸を持つ。

- **Historical L1:** 発行時 environment / policy で accepted だったか。
- **Active replay:** platform の active environment で現在も accepted か。
- **Upstream compatibility:** 指定した新 Lean / Mathlib candidate で accepted か。

新 environment で失敗したとき、元 certificate の `11.status` を書き換えてはならない。`reverification_failed` event に environment、reason、log digest を追加する。statement または proof を修正した場合は新 artifact で新 certificate を発行し、`supersedes edge` で関係を示す。

10.2 artifact retention

Full replay のため、source bundle、manifests、policy、raw axiom report、build log、必要なら build image を保持する。保持期間を短縮する場合、certificate は再現性 level を明示する。最低でも content digest と複数 location を持ち、消失を静かに無視しない。ledger は公開 export と定期 checkpoint を提供し、platform 終了後も検証可能性を残す [12]。

10.3 environment epoch の lifecycle

1. candidate epoch を content-addressed manifest として作成する。
2. golden / negative / malicious corpus を replay する。
3. active 化前に差分 report と known breakage を公開する。
4. old epoch を historical replay 用に read-only 保持する。
5. certificate ごとの migration result を append-only event として追加する。

11 testing と observability

11.1 verification corpus

Unit test だけでは L1 boundary を守れない。次の corpus を CI と production canary の両方で使う。

suite	fixture	expected invariant
Golden proofs	小さな accepted theorem と実研究例	target / axiom / digest が固定
Negative proofs	type error、未定義名、false target	決して accepted にならない
Axiom attacks	sorryAx、custom axiom、間接依存	deny / unknown を確実に拒否
Target attacks	shadowing、duplicate name、type drift	target mismatch
Target cardinality	0 / 2+ top-level theorem、unused second theorem	exactly one or typed rejection
Batch facade	3 theorem、partial failure、child retry	3 independent attempts; no batch L1
Certified imports	valid same- <i>E</i> cert、spoof、cross- <i>E</i> 、cycle、revoked cert	only valid acyclic same- <i>E</i> chain passes
Artifact attacks	newline、Unicode、symlink、path traversal	exact hash または ingest rejection
Resource attacks	loop、huge output、memory pressure	exhausted、runner recovery
Schema attacks	unknown field、duplicate key、bad Unicode	strict parser rejection
Replay matrix	architecture / clean host / independent operator	logical output 一致

Property test は canonical JSON の一意性、manifest path normalization、hash stability、status state machine の不正遷移を対象にする。production deploy は corpus の全 gate を通過しなければならない。

11.2 integrity SLO と運用 metric

可用性より integrity を優先する。推奨する初期 SLO / invariant は次である。

- accepted certificate の必須 digest / target / axiom report 完備率: **100%**、
- accepted certificate に対する corpus-based false acceptance: **0**、
- 同一 attempt が複数の terminal state を持つ件数: **0**、
- daily sampled historical replay の一致率: **100%**、
- infrastructure error、resource exhaustion、queue latency は別 metric として公開、
- artifact retrieval failure と replay unavailable を成功率から除外せず可視化。

一件でも false acceptance または artifact mismatch が確認されたら、L1 発行を停止し、key / environment / affected certificate 範囲を調査する。過去 event は削除せず incident marker と再検証結果を追加する。

12 実装 roadmap

12.1 Phase 0: claim freeze と fixture (0–30 日)

- 本書の predicate、taxonomy、schema を versioned specification として固定する。
- one user target validator と generated namespace を実装し、target count と type hash を記録する。
- multi-theorem request を独立 child job に分解する batch boundary と fixture を固定する。
- primitive axiom と certificate-backed dependency を schema / analyzer / UI で分離する。
- raw artifact bundle と environment manifest の content hashing を実装する。
- sorryAx、custom axiom、unknown parser output の negative fixture を追加する。
- UI / API の “accepted” 表示経路を棚卸しし、唯一の合成 predicate に統合する。

Gate A

Golden / negative / axiom / target corpus が CI で 100% deterministic に通り、accepted を生成できる code path が ledger-side L1 predicate の一つだけであること。未達なら外部向け certificate claim を拡張しない。

12.2 Phase 1: certificate と replay (31–60 日)

- L1 schema、canonical encoder、signature、public key discovery を実装する。
- same-environment certificate registry、content-addressed stub generator、checkpoint root を実装する。
- source / environment / policy / evidence を CAS に置き、certificate から取得可能にする。
- independent verifier CLI を別 package として作り、metadata / historical replay を提供する。
- retry を append-only attempt に変更し、terminal state の上書きを禁止する。
- staging certificate を clean host と第二 architecture で再検証する。
- valid / spoofed / cross-epoch / cyclic / revoked certificate import corpus を通す。

Gate B

無作為抽出した staging certificate を clean host で artifact から再構成でき、target identity、kernel result、axiom set が全件一致すること。取得不能、手作業修正、moving tag 依存が一件でもあれば未達とする。

12.3 Phase 2: production hardening (61–90 日)

- signer を runner から分離し、key rotation / revocation / transparency checkpoint を導入する。
- sandbox profile、egress deny、quota、archive extraction 防御を adversarial test する。
- sampled daily replay、integrity alert、issuance kill switch、incident playbook を運用する。
- environment migration の candidate corpus と append-only reverification event を実装する。
- certificate invalidation の downstream taint propagation と issuance kill switch を drill する。
- public L1 export と verifier documentation を公開し、外部 operator の replay receipt を受け付ける。

Gate C

30 日間、必須 evidence 完備率 100%、sampled replay 一致率 100%、false acceptance 0 を維持し、kill switch と key revocation drill を完了すること。これを L1 production readiness の最低条件とする。

12.4 ownership

workstream	accountable role	review separation
Kernel / target wrapper	judge maintainer	adversarial fixture reviewer
Axiom analyzer / policy	formalization maintainer	policy approver
Artifact / environment CAS	platform maintainer	reproducibility operator
Certificate / ledger / signer	security owner	key custodian + audit reviewer
Verifier CLI	independent tooling owner	must not reuse issuer trust decisions blindly
Incident response	service owner	public post-incident approver

小規模 team でも role を文書上分け、同じ人が実装する場合は時間を分けた review と公開 fixture で補う。

13 決定事項と非規範的 research questions

13.1 本仕様で固定する決定

1. L1 は kernel acceptance と operational predicate の conjunction とする。
2. Kernel status と axiom-policy status を別 field とする。

3. exact raw artifact、environment、target type を digest で拘束する。
4. 一つの L1 attempt は user-supplied top-level theorem を一つだけ持つ。
5. multi-theorem UX は L1 外部で独立 child attempt へ fan-out する。
6. accepted theorem の再利用は primitive allowlist ではなく、same-environment certificate registry で管理する。
7. logical epoch / primitive policy と append-only registry checkpoint を別に versioning する。
8. timeout、infrastructure error、unknown axiom / parser は fail-closed とする。
9. retry、migration、revocation は append-only event とし、historical result を上書きしない。
10. platform signature を replay の代替にしない。
11. target identity は pinned versioned kernel declaration / proof export encoding とし、pretty-printer を使わない。
12. canonical JSON は RFC 8785 JCS + I-JSON、signature は Ed25519、三層 envelope とする。
13. axiom closure は proof export から checker domain で再計算し、stdout parser を authority にしない。
14. registry event / checkpoint と replay objects は certificate lifetime 中保持し、two-witness cumulative log を使う。

13.2 Accepted の意味を変えない research questions

- standard axiom profile を単一にするか、constructive / classical など複数 profile を持つか。
- 必須の two-witness quorum を超えて、どの operator / jurisdiction に mirror を増やすか。
- Full-chain replay の latency を下げる content-addressed cache を、trust decision と分離してどう運用するか。
- Stable export format の次 version を Lean upstream とどう共同設計するか。
- Current taint と historical validity の差を non-expert UI でどう説明するか。

これらは Gate 0 の決定を弱めてはならない。Accepted の意味を変更する proposal は research question のまま実装せず、new schema / policy version、ADR、golden vectors、migration plan、Gate A–C の再実行を必要とする。

14 結論

L1 の価値は、強く聞こえる label ではなく、主張を小さくし、入力と環境を固定し、失敗を隠さず、誰でも replay できることにある。Lean kernel は formal derivability の強い trust anchor だが、hostile input では Comparator、proof export、official checker、independent checker と組み合わせる必要がある。その事実を公共の certificate に変えるには artifact、target、axiom、resource、ledger の境界を正確に設計する必要がある。

TheoremForces は、すべての数学的評価を L1 に押し込めるべきではない。L1 は「この一つの exact

theorem が、この exact environment、primitive axiom policy、certificate dependency chain のもとで受理された」という狭い事実を堅牢に残す。その上に provenance、migration、intent review、curation を積むことで、AI 時代の形式数学に必要な検証可能性を段階的に構築できる。

最終原則

No artifact, no L1. No exact environment, no L1. Not exactly one target, no L1. No primitive / certified dependency evidence, no L1. No Comparator and checker quorum, no L1. No independent replay path, no durable trust.

付録 A minimal judge wrapper 例

次の例は target binding の考え方を示すだけであり、production sandbox や analyzer の完全実装ではない。

```
import Mathlib
import TheoremForces.Certified.Dep_<certificate_hash>

namespace TheoremForces.Submission.Generated_01JEXAMPLE

-- The exact statement and proof body are inserted as raw artifact content.
-- This is the only user-supplied top-level theorem in the attempt.
theorem solution_target (n : Nat) : n + 0 = n := by
  simp

-- The judge resolves this fully qualified declaration, checks its type hash,
-- and records the transitive axiom report. Build success alone is insufficient.
#check TheoremForces.Submission.Generated_01JEXAMPLE.solution_target
#print axioms TheoremForces.Submission.Generated_01JEXAMPLE.solution_target

end TheoremForces.Submission.Generated_01JEXAMPLE
```

Production では user input に wrapper delimiter を破壊させず、generated name を server-side に作り、source bundle 全体を build 前に hash する。#check や #print axioms の stdout だけを根拠とせず、process exit、target identity、structured analyzer result を合成する。

付録 B state transition table

from	event	to / constraint
received	artifact hashed	environment-resolving
environment-resolving	immutable manifest found	running
running	kernel accepted	policy-checking
running	error / limit	terminal non-acceptance
policy-checking	target + primitive axioms + cert chain + evidence pass	issuable
policy-checking	deny / invalid cert / unknown / missing	terminal non-acceptance
issuable	ledger rechecks and signs	accepted
any terminal	retry requested	new attempt; original immutable
accepted	new environment replayed	append reverification event
accepted	key / artifact incident	append incident marker; never delete

付録 C 用語集

Artifact

Judge が実際に読む source bundle と manifest。UI が後から再構成した text ではない。

Axiom policy

Target の primitive axiom set を allowed / denied / unknown に分類する immutable rule set。

Certificate-backed assumption

L1 accepted theorem の exact type と certificate chain に拘束された server-generated constant。同じ logical environment からだけ import でき、primitive axiom allowlist とは別に検証する。

Certificate

Artifact、environment、target、kernel result、axiom report、run outcome を拘束する署名付き record。

Environment epoch

Lean、Mathlib、依存 package、build option、image をまとめた immutable platform environment。

Fail closed

Unknown、欠落、timeout、parser failure を成功として扱わず、受理を発行しない設計。

Kernel

Lean の core type checker。Elaborated declaration / term が型規則に従うかを検査する。

L1 Trusted Challenge、exact artifact / environment、Comparator、official + independent checker quorum、operational policy を結合した最下層 assurance。

Replay

Certificate から artifact と environment を再構成し、判定を独立に再実行すること。

Target declaration

L1 判定対象となる fully qualified Lean declaration。Name、kind、universes、kernel type / value、proof export digests で拘束する。

Trusted computing base (TCB)

L1 の正しさを支える trusted Challenge、Comparator、exporter、official / independent checker、environment、runtime、signing / ledger path の最小構成。

付録 D 参考文献と一次資料

参考文献

- [1] Lean Project, *The Lean Language Reference*, <https://lean-lang.org/doc/reference/latest/>.
- [2] Lean Project, *The Kernel*, <https://lean-lang.org/doc/reference/latest/Kernel/>.
- [3] Lean Project, *Axioms*, <https://lean-lang.org/doc/reference/latest/Axioms/>.
- [4] Lean Project, *Validating Lean Proofs*, <https://lean-lang.org/doc/reference/latest/ValidatingProofs/>.
- [5] Lean Project, *Comparator*, <https://github.com/leanprover/comparator>, accessed 2026-08-09.
- [6] A. Rundgren et al., *JSON Canonicalization Scheme (JCS)*, RFC 8785, <https://www.rfc-editor.org/rfc/rfc8785.html>.
- [7] S. Josefsson and I. Liusvaara, *Edwards-Curve Digital Signature Algorithm (EdDSA)*, RFC 8032, <https://www.rfc-editor.org/rfc/rfc8032.html>.
- [8] B. Laurie et al., *Certificate Transparency Version 2.0*, RFC 9162, <https://www.rfc-editor.org/rfc/rfc9162.html>.
- [9] Mathlib Community, *Mathlib repository*, <https://github.com/leanprover-community/mathlib4>.
- [10] TheoremForces, *Judging documentation*, <https://theoremforces.org/docs/judging>.
- [11] TheoremForces, *Security documentation*, <https://theoremforces.org/docs/security>.
- [12] TheoremForces, *Certificate ledger documentation*, <https://theoremforces.org/docs/ledger>.